Лабораторна робота №3: "Основи агентних систем: чат-бот з rag системою"

1. Загальний опис проєкту

Мета роботи — опанувати фундаментальні принципи побудови автономних AI-агентів. Ви створите інтелектуального термінального помічника, де LLM виступає не просто як чат-бот, а як «когнітивне ядро», що координує роботу наступних модулів:

- **RAG (Retrieval Augmented Generation):** Реалізація пошуку фактів у локальній базі знань. Агент зможе звертатися до завантажених документів для отримання специфічного контексту, що дозволяє надавати точні відповіді без «галюцинацій».
- **Tool Use (Function Calling):** Надання агенту можливості взаємодії із зовнішнім світом. Ви навчите модель автоматично викликати Python-функції для отримання динамічних даних або виконання розрахунків у реальному часі.
- **Persona & Memory:** Формування унікальної ролі агента та системи збереження контексту. Це забезпечить утримання заданої моделі поведінки та здатність агента пам'ятати історію взаємодії протягом усієї сесії.

2. Стек технологій та інсталяція

Усі інструменти є локальними та безкоштовними (Open Source).

Крок 1. Встановлення бібліотек (Python):

Bash

```
pip install ollama sentence-transformers scikit-learn numpy
```

Крок 2. Налаштування LLM (Ollama):

1. Завантажте Ollama.
2. Виконайте в терміналі команду для завантаження моделі (рекомендуємо `llama3.2` або `mistral`):

   ```
   ollama pull llama3.2
   ```

Крок 3. Модель Ембедінгів:
Бібліотека `sentence-transformers` автоматично завантажить модель при першому запуску скрипта.

- Рекомендована модель: `sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2` (швидка, підтримує укр/анг).

3. Вибір теми та Даних

Студент обирає **одну** тему і розвиває її.
Важливо: Лабораторна робота містить завдання, що вимагають уявлевлення про архітектуру системи (виклик зовнішніх функцій, створення бази знань). Тому рекомендуємо перед остаточним вибором теми ознайомитися із завданнями. Переконайтеся, що у вас є ідеї, як саме реалізувати необхідний функціонал (RAG, Function calling) у контексті вашого проєкту.

Приклади можливих проєктів:

- HR-Помічник: Аналіз тексту резюме, порівняння з вакансією, генерація фідбеку, симуляція співбесіди.
- Фінансовий консультант: Аналіз витрат, конвертація валют, поради щодо інвестицій та заощаджень.
- Support Chatbot: Автоматизована підтримка продукту (пошук по технічній документації) з можливістю створення тікетів.
- AI Dungeon Master (RPG): Текстова рольова гра з генерацією сюжету, де RAG зберігає "лор" світу, а інструменти відповідають за ігрову механіку (кидання кубиків, інвентар).
- Юридичний навігатор: Пошук по базі законів та роз'яснення складних термінів "простою мовою".

- Персональний нутриціолог: Розрахунок калорій (математична функція), генерація рецептів під інгредієнти користувача.
- AI Тьютор (Точні науки): Допомога у вивченні математики. Агент має перевіряти розв'язки та пояснювати теореми, використовуючи перевірені джерела.
- E-commerce Concierge: Підбір товарів за складними, нечіткими критеріями ("подарунок для друга, який любить гори, бюджет до 1000 грн").

4. Алгоритм виконання (Покроково)

Етап 1: Підготовка основи (Basic Terminal Chatbot)

Створіть базовий скрипт `main.py`, який забезпечує спілкування з LLM у циклі.

```
import ollama

print("\n--- Базовий чатбот запущений (напишіть 'exit' для ви
ходу) ---")

while True:
    user_input = input("\nUser: ")
    if user_input.lower() in ["exit", "quit"]:
        break

    # Простий виклик моделі без контексту
    try:
        response = ollama.chat(
            model="llama3",
            messages=[{"role": "user", "content": user_inpu
t}]
        )
        print(f"AI: {response['message']['content']}")
    except Exception as e:
        print(f"Помилка: {e}")
```

Етап 2: Системний промпт (Persona Definition)

Задайте поведінку моделі. Вона має знати, хто вона (наприклад, "Ти — помічник кафедри прикладної математики"). Це робиться через додавання повідомлення з роллю `system` на початку діалогу.

Код-сніпет:

```
import ollama

# Визначаємо роль асистента
SYSTEM_PROMPT = """
Ти — корисний асистент кафедри прикладної математики.
Ти допомагаєш студентам з питаннями щодо курсів, дедлайнів та
оцінок.
Будь ввічливим та конкретним у відповідях.
"""

print("\n--- Чатбот з персоною запущений (напишіть 'exit' для
виходу) ---")

while True:
    user_input = input("\nUser: ")
    if user_input.lower() in ["exit", "quit"]:
        break

    # Додаємо системний промпт на початку
    messages = [
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user", "content": user_input}
    ]

    try:
        response = ollama.chat(
            model="llama3",
            messages=messages
        )
```

```
        print(f"AI: {response['message']['content']}")
    except Exception as e:
        print(f"Помилка: {e}")
```

Етап 3: Підготовка індексу (Data Ingestion)

Створіть окремий скрипт `create_db.py`. Він перетворює текстові факти на вектори (ембеддінги) і зберігає їх у JSON-файл. Це ваша «довготривала пам'ять» (Knowledge Base).

```
import json
from sentence_transformers import SentenceTransformer

# 1. Ваші факти (Knowledge Base)
data = [
    {"text": "Дедлайн здачі курсової роботи - 25 травня."},
    {"text": "Консультації проходять щосереди о 14:00 в 205 а
удиторії."},
    {"text": "Максимальний бал за лабораторну роботу - 15 бал
ів."},
    {"text": "Викладач курсу - професор Тьюрінг."},
    {"text": "Для допуску до іспиту потрібно набрати мінімум
60 балів."}
]

# 2. Завантаження моделі (використовуємо легку модель)
print("Завантаження моделі ембеддінгів...")
model = SentenceTransformer('sentence-transformers/all-MiniLM
-L6-v2')

# 3. Генерація векторів
print("Генерую вектори...")
for item in data:
    item['vector'] = model.encode(item['text']).tolist()

# 4. Збереження у файл
```

```
with open('knowledge_base.json', 'w', encoding='utf-8') as f:
    json.dump(data, f, ensure_ascii=False, indent=2)

print("Базу знань 'knowledge_base.json' успішно створено!")
```

Етап 4: Інтеграція RAG (Retrieval System)

Модифікуйте `main.py`. Перед відправкою запиту до LLM, скрипт має знайти найбільш релевантний факт у `knowledge_base.json` (використовуючи косинусну подібність) і додати його у контекст.

```
import json
import numpy as np
from sentence_transformers import SentenceTransformer
import ollama

# Завантаження бази знань та моделі
print("Завантаження бази знань та моделі...")
with open('knowledge_base.json', 'r', encoding='utf-8') as f:
    knowledge_base = json.load(f)
embedding_model = SentenceTransformer('sentence-transformers/
all-MiniLM-L6-v2')


def retrieve_info(query, threshold=0.3):
    """Пошук найбільш релевантного факту"""
    query_vec = embedding_model.encode(query)
    best_score = -1
    best_text = None

    for item in knowledge_base:
        vec = np.array(item['vector'])
        # Косинусна подібність
        score = np.dot(query_vec, vec) / (np.linalg.norm(quer
y_vec) * np.linalg.norm(vec))
        if score > best_score:
```

```
            best_score = score
            best_text = item['text']

    if best_score > threshold:
        return f"[Контекст із бази знань]: {best_text}"
    return ""


SYSTEM_PROMPT_TEMPLATE = """
Ти — корисний асистент кафедри прикладної математики.
Ти допомагаєш студентам з питаннями щодо курсів, дедлайнів та
оцінок.
Будь ввічливим та конкретним у відповідях.
{rag_context}
"""

print("\n--- Чатбот з RAG запущений (напишіть 'exit' для вихо
ду) ---")

while True:
    user_input = input("\nUser: ")
    if user_input.lower() in ["exit", "quit"]:
        break

    # Пошук релевантної інформації
    rag_context = retrieve_info(user_input)
    if rag_context:
        print(f"(Знайдено контекст: {rag_context})")

    # Формування промпту з контекстом
    system_prompt = SYSTEM_PROMPT_TEMPLATE.format(rag_context
=rag_context)
    messages = [
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_input}
    ]
```

```
    try:
        response = ollama.chat(
            model="llama3",
            messages=messages
        )
        print(f"AI: {response['message']['content']}")
    except Exception as e:
        print(f"Помилка: {e}")
```

Етап 5: Реалізація функцій-інструментів (Tools Definition)

Визначте Python-функції, які агент зможе викликати.

```
# tools.py
import random

def get_current_weather(city):
    """Повертає погоду (імітація)"""
    # Імітуємо різні умови для різних міст
    weather_data = {
        "Lviv": ("Дощ", 12),
        "Kyiv": ("Сонячно", 20),
        "Odesa": ("Вітряно", 18)
    }

    # Випадковий вибір, якщо міста немає в базі
    condition, temp = weather_data.get(city, (random.choice
(["Хмарно", "Ясно"]), random.randint(10, 30)))
    return f"У місті {city} зараз {condition}, температура +
{temp}°C"

def calculate_grade(lab_points):
    """Розрахунок підсумкової оцінки. Очікує число або рядок,
що конвертується в число."""
    try:
```

```
        points = int(lab_points)
        # Логіка: оцінка множиться на коефіцієнт 1.2
        return f"Загальний бал: {points * 1.2}"
    except ValueError:
        raise ValueError("Аргумент має бути числом (наприкла
д, 10)")

# Реєстр доступних функцій
available_tools = {
    "get_weather": get_current_weather,
    "calc_grade": calculate_grade
}
```

Етап 6:

Оновіть системний промпт, пояснивши моделі, що вона має доступ до інструментів. Якщо модель повертає JSON із назвою функції, скрипт має її виконати.
Ви маєте чітко пояснити моделі формат виклику. Наприклад: `FUNCTION_NAME(arg1, arg2)`

```
import json
import re
import numpy as np
from sentence_transformers import SentenceTransformer
import ollama
from tools import available_tools

# Завантаження ресурсів
print("Завантаження бази знань та моделі...")
with open('knowledge_base.json', 'r', encoding='utf-8') as f:
    knowledge_base = json.load(f)
embedding_model = SentenceTransformer('sentence-transformers/
all-MiniLM-L6-v2')
```

```
def retrieve_info(query, threshold=0.3):
    """Пошук релевантного факту"""
    query_vec = embedding_model.encode(query)
    best_score = -1
    best_text = None

    for item in knowledge_base:
        vec = np.array(item['vector'])
        score = np.dot(query_vec, vec) / (np.linalg.norm(quer
y_vec) * np.linalg.norm(vec))
        if score > best_score:
            best_score = score
            best_text = item['text']

    if best_score > threshold:
        return f"[Контекст]: {best_text}"
    return ""


def execute_tools_in_text(text):
    """Шукає та виконує виклики функцій у форматі function_na
me(args)"""
    pattern = r"(\w+)\((.*?)\)"
    matches = re.findall(pattern, text)

    if not matches:
        return text

    new_text = text
    for func_name, args_str in matches:
        if func_name in available_tools:
            print(f" >> Виклик: {func_name}({args_str})")
            try:
                args = [arg.strip() for arg in args_str.split
(',') if arg.strip()]
                result = available_tools[func_name](*args)
```

```
                new_text = new_text.replace(f"{func_name}({ar
gs_str})", str(result))
            except Exception as e:
                new_text = new_text.replace(
                    f"{func_name}({args_str})",
                    f"[Помилка: {str(e)}]"
                )
    return new_text


SYSTEM_PROMPT_TEMPLATE = """
Ти — корисний асистент кафедри прикладної математики.
Ти маєш доступ до інструментів. Якщо потрібно дізнатися погод
у або порахувати оцінку,
НАПИШИ виклик функції прямо у відповіді у форматі:
get_weather(City)
calc_grade(Points)
Вони не потрібні користувачу, лише для тебе, якщо ти хочеш ви
вести такого роду інформацію, їх виклик буде замінено на резу
льтат.

{rag_context}
"""

print("\n--- Агент з інструментами запущений (напишіть 'exit'
для виходу) ---")

while True:
    user_input = input("\nUser: ")
    if user_input.lower() in ["exit", "quit"]:
        break

    rag_context = retrieve_info(user_input)
    system_prompt = SYSTEM_PROMPT_TEMPLATE.format(rag_context
=rag_context)
```

```
    messages = [
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_input}
    ]

    try:
        response = ollama.chat(
            model="llama3",
            messages=messages,
            options={"temperature": 0.0}
        )
        ai_response = response['message']['content']

        # Виконуємо інструменти
        final_response = execute_tools_in_text(ai_response)
        print(f"AI: {final_response}")

    except Exception as e:
        print(f"Помилка: {e}")
```

Етап 7: Пам'ять та Self-Correction (Advanced)

Додайте список `history`, щоб зберігати контекст розмови. Обмежте його останніми 2 парами "питання-відповідь", щоб не переповнювати контекстне вікно.

Додатково (Self-Correction): Якщо інструмент повернув помилку, автоматично попросіть модель виправити параметри виклику.

```
import json
import re
import numpy as np
from collections import deque
from sentence_transformers import SentenceTransformer
import ollama
from tools import available_tools
```

```
# --- КОНФІГУРАЦІЯ ---
# Налаштування пам'яті (Крок 7: Ковзне вікно)
history = deque(maxlen=4)  # Зберігає останні 2 пари запитань
-відповідей

# --- ЗАВАНТАЖЕННЯ РЕСУРСІВ (Крок 3 та 4) ---
print("Завантаження бази знань та моделі...")
try:
    with open('knowledge_base.json', 'r', encoding='utf-8') a
s f:
        knowledge_base = json.load(f)
    embedding_model = SentenceTransformer('sentence-transform
ers/all-MiniLM-L6-v2')
except FileNotFoundError:
    print("ПОМИЛКА: Спочатку запустіть create_db.py!")
    exit()

# --- ФУНКЦІЇ ---


def retrieve_info(query, threshold=0.3):
    """Крок 4: RAG - Пошук схожих фактів"""
    query_vec = embedding_model.encode(query)
    best_score = -1
    best_text = None

    for item in knowledge_base:
        vec = np.array(item['vector'])
        # Косинусна схожість
        score = np.dot(query_vec, vec) / (np.linalg.norm(quer
y_vec) * np.linalg.norm(vec))
        if score > best_score:
            best_score = score
            best_text = item['text']
```

```
    if best_score > threshold:
        return f"[Знайдено в базі знань]: {best_text}"
    return ""


def execute_tools_in_text(text):
    """Крок 6: Пошук шаблонів та виконання"""
    # Шукає pattern: назва_функції(аргументи)
    pattern = r"(\w+)\((.*?)\)"
    matches = re.findall(pattern, text)

    if not matches:
        return text, False  # Текст без змін, помилок немає

    new_text = text
    error_occurred = False

    for func_name, args_str in matches:
        if func_name in available_tools:
            print(f" >> Виклик інструменту: {func_name} з арг
ументами: {args_str}")
            try:
                # Очистка аргументів та виклик
                args = [arg.strip() for arg in args_str.split
(',')]
                # Якщо аргумент порожній (наприклад func()),
передаємо порожній список
                if len(args) == 1 and args[0] == "": args =
[]

                result = available_tools[func_name](*args)

                # Заміна виклику на результат в тексті
                full_match = f"{func_name}({args_str})"
                new_text = new_text.replace(full_match, str(r
esult))
```

```
            except Exception as e:
                # Крок 7: Тригер самокорекції
                error_msg = f"[Помилка при виклику {func_nam
e}: {str(e)}]"
                new_text = new_text.replace(f"{func_name}({ar
gs_str})", error_msg)
                error_occurred = True
        else:
            # Функцію не знайдено, ігноруємо або помічаємо
            pass

    return new_text, error_occurred

# --- ОСНОВНИЙ ЦИКЛ (Крок 1 та 7) ---


system_prompt_template = """
Ти — корисний асистент кафедри прикладної математики.
Ти маєш доступ до інструментів. Якщо потрібно дізнатися погод
у або порахувати оцінку,
НАПИШИ виклик функції прямо у відповіді у форматі:
get_weather(City)
calc_grade(Points)
Вони не потрібні користувачу, лише для тебе, якщо ти хочеш ви
вести такого роду інформацію, їх виклик буде замінено на резу
льтат.

Контекст із бази знань: {rag_context}

Історія діалогу з користувачем (останнє зверху):
{history}
"""

print("\n--- Агент запущений (напишіть 'exit' для виходу) ---
")
```

```
while True:
    user_input = input("\nUser: ")
    if user_input.lower() in ["exit", "quit"]: break

    # 1. RAG пошук
    rag_context = retrieve_info(user_input)
    if rag_context:
        print(f"(Debug: RAG додав контекст -> {rag_contex
t})")

    # 2. Формування промпту
    current_system_prompt = system_prompt_template.format(rag
_context=rag_context, history=history)

    messages = [{"role": "system", "content": current_system_
prompt}]
    messages.extend(history)  # Додаємо історію (пам'ять)
    messages.append({"role": "user", "content": user_input})

    # 3. Виклик LLM (Перша спроба)
    try:
        response = ollama.chat(
            model="llama3",
            messages=messages,
            options={"temperature": 0.0}  # Для більш точних
викликів функцій
        )
        ai_raw_response = response['message']['content']

        # 4. Обробка інструментів та самокорекція
        final_response, has_error = execute_tools_in_text(ai_
raw_response)

        # Крок 7: Самокорекція (спрощена версія)
        if has_error:
```

```
            print("!(Self-Correction): Виявлено помилку, прош
у модель виправити...")
            # Додаємо помилковий вивід в історію і просимо ви
правити
            messages.append({"role": "assistant", "content":
ai_raw_response})
            messages.append({"role": "system", "content": f"В
иникла помилка при виконанні інструменту. Результат: {final_r
esponse}. Спробуй виправити аргументи або поясни помилку."})

            # Повторний запит
            retry_response = ollama.chat(
                model="llama3",
                messages=messages
            )
            final_response = retry_response['message']['conte
nt']

        print(f"AI: {final_response}")

        # 5. Оновлення пам'яті
        history.append({"role": "user", "content": user_inpu
t})
        history.append({"role": "assistant", "content": final
_response})

    except Exception as e:
        print(f"Error: {e}")
```

5. Критерії оцінювання (15 балів)

Основа (10 балів):

1. **RAG-система:** Працює пошук по JSON-файлу за змістом.
2. **Function Calling:** Агент коректно визначає, коли треба викликати інструмент, і отримує результат (навіть якщо це random-функція).

3. **Працюючий застосунок:** Код структурований, запускається без помилок, реалізовано цикл діалогу.

Додаткові фічі (5 балів):

1. **[2 бали] Persona & Guardrails:**
   - Чіткий `System Prompt` (роль).
   - Бот відмовляється відповідати на питання не по темі (наприклад, юрист не дає медичних порад).
2. **[1 бал] Пам'ять:**
   - Бот пам'ятає контекст розмови (реалізовано передачу останніх 2 пар питання-відповідь у промпт).
3. **[2 бали] Self-Correction:**
   - Реалізовано поріг схожості (Threshold). Якщо `similarity_score < 0.3` (або інше число), бот каже "Немає інформації в базі", замість того щоб вигадувати.